

Real NYC taxi data

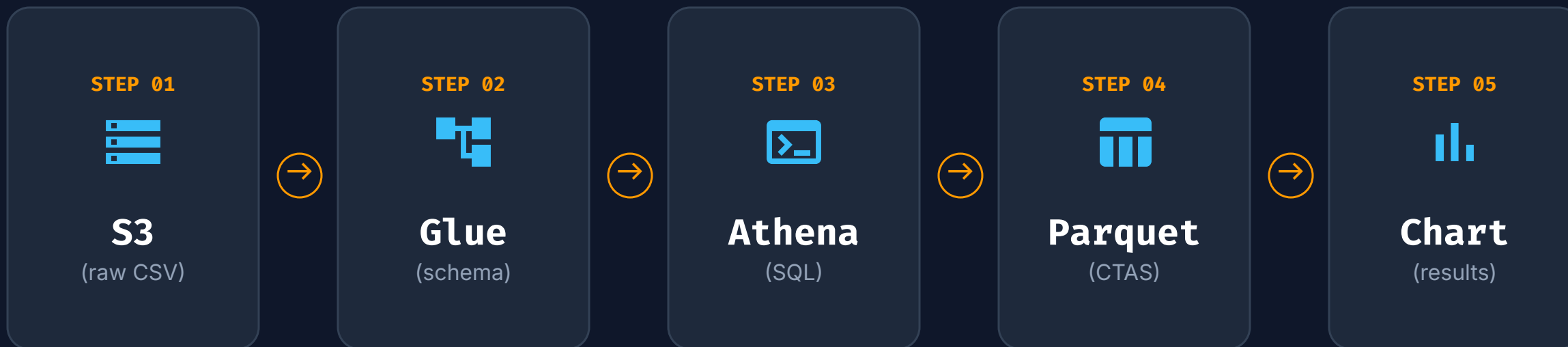
AWS for Data Analytics: Your First Cloud Data Pipeline

You will build a working serverless analytics pipeline today. No servers, no installs, no database admin.

END-TO-END PIPELINE ARCHITECTURE

What we're building

A serverless analytics workflow transforming raw CSV data into interactive visual insights.



- Serverless Architecture: Zero server management & pay-per-query execution

Ground rules

- **One region: Asia Pacific (Mumbai)** . Set it now, never change it.
- Wrong region = bucket vanishes and every click is "Access Denied."
- Do the cleanup at the end. It is not optional.
- Interrupt with questions at any time.

Differences between Data & Information



Data



Information

Cloud & AWS Foundations

●(you are here: before the pipeline)



What is cloud computing?

Renting a car vs owning one

Owning

Buy it upfront, insure it, service it, park it.

It sits idle 95% of the time.

Renting (Cloud)

Pick it up when needed, pay for hours used, hand back keys.

Rent compute and storage by the second from someone else's data centre.

You never touch hardware. You never wait for a procurement cycle.

On-prem pain vs cloud benefits

On-premises

Buy servers upfront (capex)

Guess capacity 3 years ahead

You patch, cool, and replace it

Weeks to provision

Idle capacity is wasted money

Cloud

Pay as you go (opex)

Scale up and down on demand

Provider handles the hardware

Minutes to provision

Idle costs nothing

On-Premises

9%

Software Licenses

Customisation &
Implementation

Hardware

IT Personnel

Maintenance

Training

Ongoing Costs

- Apply Fixes, Patches, Upgrade
- Downtime
- Performance tuning
- Rewrite customizations
- Rewrite integrations
- Upgrade dependent applications
- Ongoing burden on IT
- Maintain/upgrade hardware
- Maintain/upgrade network
- Maintain/upgrade security
- Maintain/upgrade database

Cloud Computing

68%

Subscription Fee

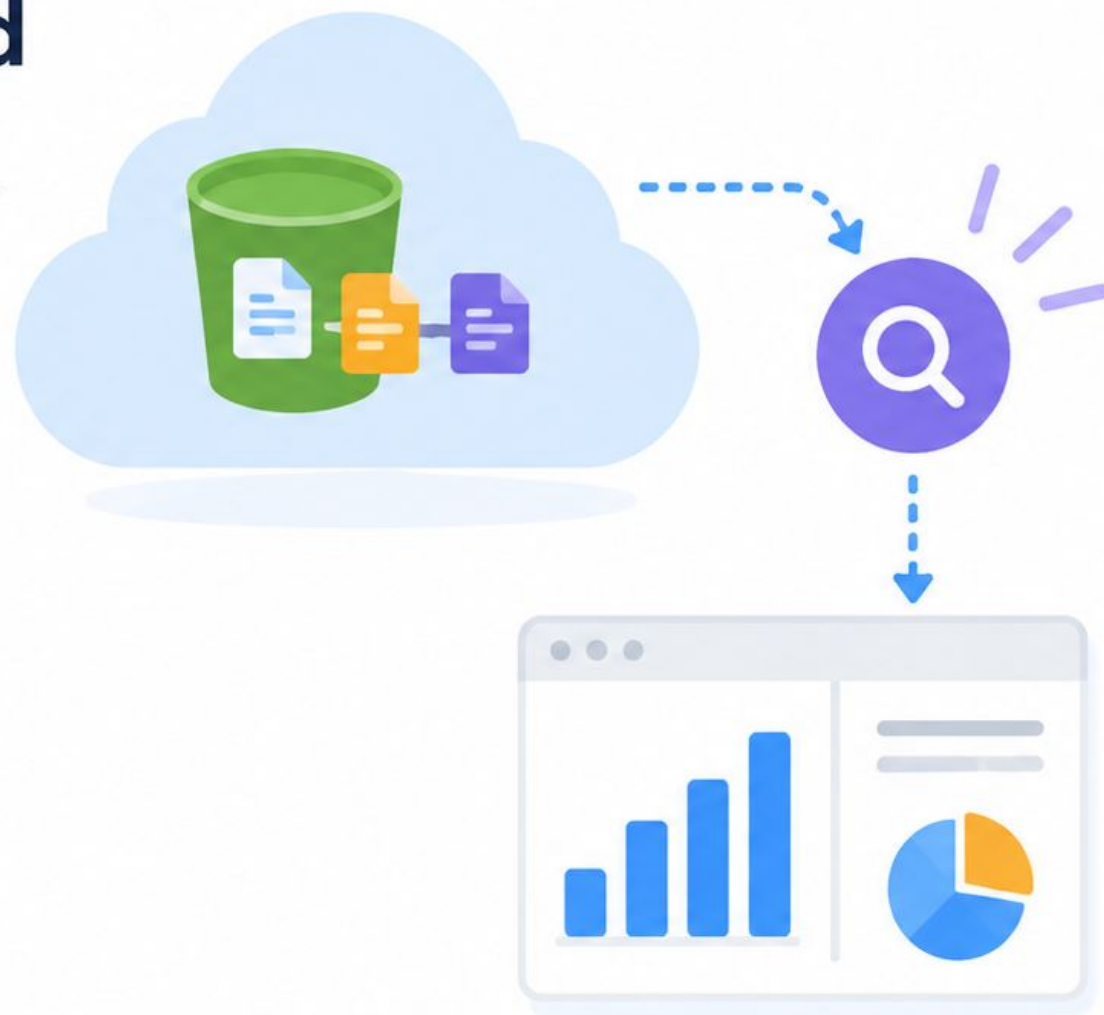
Implementation,
Customisation &
Training

Ongoing Costs

- Subscription fee

Why analytics loves the cloud

- **Store cheap.** Object storage costs cents per GB per month. Keep everything.
- **Query on demand.** Spin up query capacity for 2 seconds, pay for 2 seconds.
- **Decouple storage from compute.** The data stays put; many engines read it.
- **No "import the data first."** Query files where they already live.



The AWS map: Regions & Availability Zones

Region

A geographic area.

Example: (Mumbai), (N. Virginia).

Availability Zone

One or more isolated data centres sitting inside a Region.

Your resources live in a chosen Region. The console shows only that Region.

Choose a Region for latency, cost, and data-residency rules.

The console and the three services we need



Amazon S3

Where the files live. Infinite object storage.



AWS Glue

The catalogue. Figures out the columns in your files.



Amazon Athena

The SQL engine. Serverless queries over S3.

That's it. AWS has 200+ services; today we use three.

IAM in one slide

- **User** = a person or app with credentials. Today you sign in as .
- **Role** = a set of permissions a service can assume. Our crawler uses .
- **Policy** = the JSON that says who can do what to which resource.
- **Least privilege** = grant the minimum needed, nothing more.



How AWS billing works

- You pay per unit consumed: GB stored, GB scanned, requests made.
- **Free Tier** covers meaningful amounts of S3 for new accounts.
- Costs come from things left running, not from things you did once.
- **Budgets** send you an email before the bill surprises you.

Athena's cost model

You pay per byte of data scanned — roughly \$5 per TB.

Scan less data → pay less..

Set your region

1. Sign in with the lab URL as user `student`
2. Top-right region selector → **Asia Pacific (Mumbai)**`ap-south-1`

Expected result:

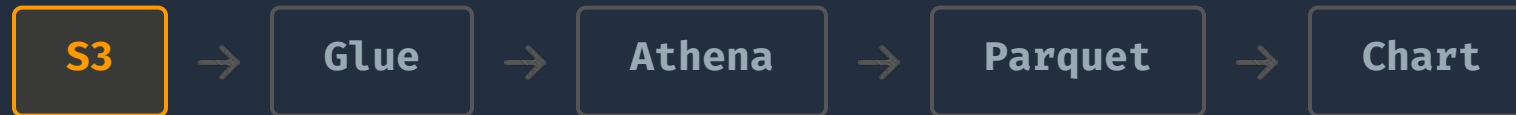
Region reads "Mumbai" in the top-right,.

Recap

- Cloud = rent instead of own.
- Regions are geographic; your console view is region-scoped.
- Three services today: S3, Glue, Athena.
- IAM: users for people, roles for services, least privilege always.
- Athena bills per byte scanned.

HOUR 2

Storage & Ingestion: Amazon S3



(you are here: stage 1)

What is object storage?

An infinite Google Drive with an API

- You store **objects** (files) and get back a **key** (the path).
- No disks to size, no volumes to grow. It just keeps accepting files.
- Built for durability: data is replicated across facilities automatically.
- Any size file, from 1 byte to terabytes.



Buckets, objects, prefixes

Bucket

The top-level container. Globally unique name.

`aws-analytics-lab-student01-`

Object

One file plus its metadata.

Prefix

The part of the key before the last `.` *It looks like a folder.*

There are no real folders in S3. `is one flat key.`

What is a data lake?

- A single place where raw data lands, in open file formats.
- **Schema-on-read:** decide structure when querying, not when loading.
- A data warehouse demands structure up front. A lake accepts first, structures later.
- S3 is the storage layer of most data lakes on AWS.

The dataset: NYC Yellow Taxi

Real, famous, and published monthly by the TLC

— 20,000 trips, 14 columns, 1.7 MB

trip_id, tpep_pickup_datetime, tpep_dropoff_datetime, passenger_count, trip_distance, pu_location_id, do_location_id, payment_type, fare_amount, tip_amount, tolls_amount, mta_tax, improvement_surcharge, total_amount

— 51 zones

LocationID, Borough, Zone

— 4 types

1 Credit card • 2 Cash • 3 No charge • 4 Dispute

Create folders and upload CSVs

1. S3 → open `.aws-analytics-lab-student01-`
2. Create folder `.` Inside it create `,, . trips zones payments`
3. Upload → `ellow_taxi_sample.csv` `raw/trips/`
4. Upload → `zone_lookup.csv` `raw/zones/`
5. Upload → `payment_type_lookup.csv` `raw/payments/`
- .

Expected result:

```
raw/payments/payment_type_lookup.csv 74 Bytes raw/trips/yellow_taxi_sample.csv 1.7 MiB
raw/zones/taxi_zone_lookup.csv 1.5 KiB
```

Why one folder per file type?

Crawler rule: one folder = one table

- Point a crawler at and it walks the sub-folders.
- → table . → table .
- Put all CSVs in the same prefix = **one messy table** with mismatched columns.
- This is the most common mistake in this lab.

Recap

- S3 stores objects in buckets; prefixes only look like folders.
- A data lake lands raw data first and applies schema at read time.
- Layout: `raw/` `processed/` `athena-results/`
- One folder per file type, because one folder becomes one table.

Catalog & Query: Glue + Athena



(you are here: stages 2 and 3)

The problem

Your files have no schema

- S3 sees three CSVs. It does not know they have columns.
- SQL needs names and types: is text or a number?
- Writing schema by hand = 14 columns × 3 tables, typed manually.
- We need something to read the files and work it out for us.

AWS Glue Data Catalog + Crawler

- **Data Catalog** = a metastore. Describes where data lives and what's in it.
- **Crawler** = a job that infers columns and writes tables to Catalog.
- The Catalog holds **metadata only**. Your data never leaves S3.
- Athena, Redshift, and EMR all read this same Catalog.

Think of it as the library index, not the books.

Create and run the crawler

1. Glue → Crawlers → **Create crawler**. Name:
2. Data source → S3 → **Crawl all sub-folders**
3. IAM role → choose existing →
4. Target database → . Table prefix: blank.
5. Create, then **Run crawler**. Wait about 40 seconds.
- .

Expected result:

Status , **tables created: 3**

LocationId as . Glue lowercases column names.

[RESULT]

What the crawler produced

Table	Location	Format
trips	raw/trips/	CSV
zones	raw/zones/	CSV
payments	raw/payments/	CSV

Schema of (inferred, not typed by hand):

```
trip_id bigint • trip_distance double • pu_location_id bigint • fare_amount double • total_amount double
...
```

Two things to notice

- Real column names, not → CSV header detected.

What is Amazon Athena?

Serverless SQL directly on S3

- Standard SQL (Trino engine) over files sitting in S3.
- Nothing to provision, patch, or start. Open the editor and query.
- Reads the table definitions the crawler wrote into the Catalog.
- Results written as CSV to automatically.

Athena vs a traditional database

Traditional database

Load the data in first

Server runs 24/7

Pay for uptime

Storage tied to compute

You size and tune it

Athena

Query files in place

No server at all

Pay per byte scanned

Storage is just S3

Scales automatically

Trade-off: Athena is built for analytical scans, not single-row lookups or updates.

>_ [HANDS-ON] First queries

```
-- Q1 Peek at the data SELECT * FROM trips LIMIT 10; -- Q2 How many trips? SELECT  
COUNT(*) AS total_trips FROM trips; -- Q3 Average fare and distance SELECT  
ROUND(AVG(trip_distance), 2) AS avg_distance_miles, ROUND(AVG(fare_amount), 2) AS  
avg_fare FROM trips;
```

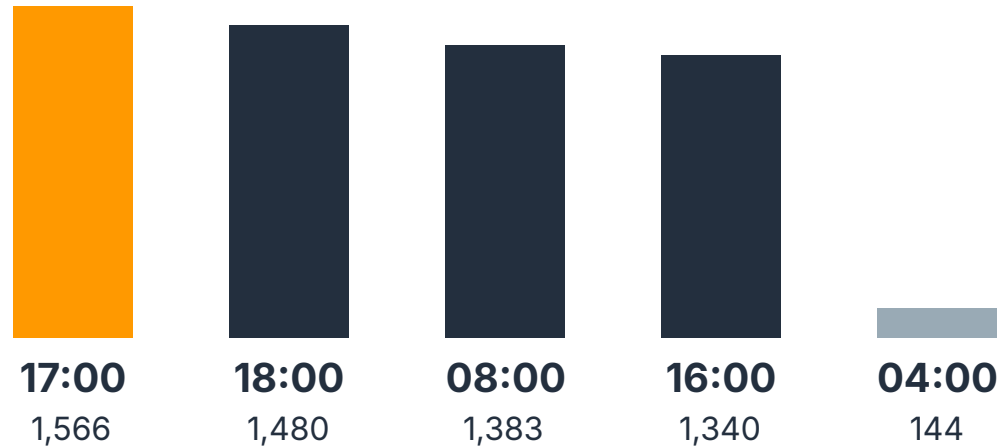
Expected results:

Q2 → 20000 · **Q3** → 4.65 miles, \$21.77

[RESULT] Q4

Busiest pickup hours

```
SELECT HOUR(CAST(tpep_pickup_datetime AS timestamp)) AS pickup_hour, COUNT(*) AS trips FROM trips GROUP BY 1 ORDER BY trips DESC;
```



Evening rush leads, morning rush follows, 4 AM is dead.

[RESULT] Q5 & Q6 — JOINS bring the lookups in

Q5 Tips by payment type

payment_desc	trips	avg_tip
Credit card	8,439	3.28
Cash	5,791	0.00
No charge	2,935	0.00
Dispute	2,835	0.00

Q6 Trips by pickup borough

borough	trips	avg_total
Manhattan	14,392	25.28
Queens	3,725	25.23
Brooklyn	1,013	25.20
Bronx	522	25.01

Only card trips show a tip.
That is data semantics, not a bug — cash tips never reach the meter.

Recap

- Queried data in place. No database server exists.
- Crawler turned three CSVs into queryable tables.
- Catalog stores metadata; data never moved.
- Standard SQL works directly on files in S3.
- Column names are lowercase (Glue lowercases them).

HOUR 4

Deeper Analytics + The Big Picture



(you are here: stages 4 and 5)

[RESULT] Q7 & Q8

Aggregation and filtering

```
-- Q8 Long trips only: WHERE filters rows, HAVING filters groups SELECT z.zone, COUNT(*) AS long_trips,  
ROUND(AVG(t.trip_distance), 2) AS avg_dist FROM trips t JOIN zones z ON t.pu_location_id = z.locationid WHERE t.trip_distance  
> 10 GROUP BY z.zone HAVING COUNT(*) > 20 ORDER BY long_trips DESC;
```

42 zones clear the bar. JFK Airport 218 · Midtown North 214 · LaGuardia 172.

Airports and Midtown dominate long trips — exactly what you'd expect.

Q7 (Revenue top row): Manhattan / Credit card — 6,078 trips, \$165,498.01.

Total revenue sample: \$505,294.62.

What is ETL?

Extract → Transform → Load

- **Extract** — read raw CSVs from .
- **Transform** — aggregate, join, clean, reshape.
- **Load** — write result somewhere optimised for reading.

Raw stays untouched. Processed is cheap and fast to query.

You did E and T all morning. Now you'll do L.

CTAS: CREATE TABLE AS SELECT

Run a query, save the answer back to S3

- One statement executes SELECT, writes files, registers new table.
- Output format: **Parquet** — columnar and compressed.
- Location: our `processed/` prefix
- Result: a real table you can query, plus real files you own.

</> [HANDS-ON] Q9

Build daily_revenue_summary

```
CREATE TABLE daily_revenue_summary WITH ( format = 'PARQUET', external_location =  
's3://aws-analytics-lab-student01-/processed/daily_revenue/' ) AS SELECT DATE(CAST(tpep_pickup_datetime AS timestamp))  
AS trip_date, COUNT(*) AS trips, ROUND(SUM(total_amount), 2) AS revenue, ROUND(AVG(tip_amount), 2) AS avg_tip FROM  
trips GROUP BY DATE(CAST(tpep_pickup_datetime AS timestamp));
```

Expected result:

No rows returned (creates, doesn't select).

S3 → — a 1.3 KiB Parquet file exists.

[RESULT]

The cost lesson, measured

Query	Source	Bytes scanned
COUNT(*) FROM trips	CSV	1,773,908
SUM(total_amount)	CSV	1,773,908
SUM(revenue)	Parquet summary	244

~7,270× less data scanned for the same answer

Both paths returned \$505,294.62. Processed layer and raw layer agree.

CSV is row-oriented. Parquet is columnar.

[HANDS-ON] Make a chart

No BI subscription needed

1. Run `SELECT * FROM daily_revenue_summary ORDER BY trip_date;`
2. Results panel → **Download results (CSV)**.
3. Open Excel or Google Sheets → select data → Insert → Chart.
4. Line chart for daily revenue.
- .

Expected result:

A chart tracking revenue across January 2025.

This scales

Today

20,000 rows

1.7 MB

Real TLC Dataset

Billions of trips

Since 2009

The SQL does not change. Not one character.

What changes is file format, partitioning, and query discipline.

Cost control habits

- Select columns you need. Never ~~✗~~ on a large table.
- Use while exploring.
- Partition by date or region so queries skip whole folders.
- Convert to Parquet/ORC. Columnar + compressed = fewer bytes.
- Set a workgroup scan limit. Ours refuses > 200 MB.

Where analytics goes next

(awareness only — not today)

- **Amazon Redshift** — warehouse for heavy, repeated BI workloads.
- **Amazon Kinesis / MSK** — streaming data instead of files at rest.
- **AWS Glue ETL / EMR** — Spark jobs for massive transformations.
- **Amazon QuickSight** — managed dashboards over the Catalog.
- **Amazon SageMaker** — ML on the data you just catalogued.

All of them read the exact same S3 data lake you built today.

CLEANUP — do this before you leave

This is the most important slide of the hour

- 1 Athena: **DROP TABLE IF EXISTS daily_revenue_summary;**
- 2 S3: Open the bucket → Empty → type *permanently delete* → Empty.
- .

This clears , ~~raw/~~ , **processed/** **athena-results/**

(Optional) Glue: delete crawler and tables.

Step 2 is manual and mandatory.

The lab cannot be torn down while the bucket still holds files.

Troubleshooting quick reference

Symptom	Cause	Fix
"Access denied" / bucket missing	Wrong region	Switch to Mumbai ap-south-1
Crawler won't let you make a role	Lab user can't create roles	Pick existing role
Columns named col0, col1	Header not detected	Re-run crawler
One messy combined table	All CSVs in one prefix	One file type per sub-folder
Athena: "no output location"	Workgroup not selected	Select analytics-lab-student01
CTAS: "location already exists"	Target prefix has objects	Delete processed/ prefix
Join on locationid fails	Glue lowercases columns	Use lowercase everywhere

What you built today

S3 → Glue → Athena → CTAS (Parquet) → Chart

- Landed raw CSVs in a structured data lake layout.
- Auto-generated schemas for 3 tables — zero DDL typed by hand.
- Ran 9 analytical queries including multi-table JOINS and GROUP BY.
- Built processed Parquet layer. Proved 7,270× scan reduction.
- Charted the result and cleaned up after yourself.

No server was provisioned, started, patched, or paid for by the hour.

Where to go next

- **Practice:** rerun on real TLC monthly Parquet public dataset.
- **Learn:** AWS Skill Builder — Data Analytics Fundamentals.
- **Read:** Athena user guide, Glue developer guide.
- **Build:** add partitioning by `trip_date` and re-measure scans.
- **Certify:** AWS Certified Data Engineer – Associate.

Mini-assessment — try these yourself

Which borough has the highest average tip?

Brooklyn, 1.43

What is the busiest single hour?

17:00 — 1,566 trips

Total revenue from credit card trips?

229,417.10

Questions?

Thank you

You now know how to store, catalogue, query, transform, and visualise data in the cloud — and how to keep the bill small while doing it.